DEPARTMENT OF FORENSIC SCIENCE

CYBER RANGERS

ACTIVITY- 2

Live Threat Intelligence Feed Simulator with AI-Based Detection and Automated Firewall

SUBMITTED BY:

ABHISHEK M S
24MSRDF002
M.Sc DIGITAL FORENSICS AND
INFORMATION SECURITY

SUBMITTED TO:

N VISHNU VENKATESH
ASSISTANT PROFESSOR
DEPARTMENT OF FORENSIC SCIENCE
JAIN UNIVERSITY SCHOOL OF SCIENCE

Live Threat Intelligence Feed Simulator with AI-Based Detection and Automated Firewall

Introduction

In today's digital world, cyber threats such as malware attacks, phishing attempts, and unauthorized access are increasing rapidly. Traditional security systems rely on predefined rules and often fail to detect new or unknown threats. Therefore, modern cybersecurity solutions focus on **real-time monitoring, threat intelligence, and proactive Defense mechanisms**.

Threat Intelligence systems collect and analyse information about potential cyber threats from various sources. These systems help organizations detect malicious activities early and respond effectively.

This project implements a **Live Threat Intelligence Feed Simulator**, which mimics a real-world cybersecurity monitoring system. It integrates external threat intelligence data, simulates network traffic, detects malicious IP addresses, and automatically blocks them using a firewall mechanism. Additionally, it uses **Artificial Intelligence (AI)** to identify abnormal traffic patterns and generate alerts.

Objectives

The main objectives of this project are:

- To simulate real-time network traffic
- To integrate external threat intelligence feeds (AbuseIPDB API)
- To detect malicious IP addresses from incoming traffic
- To implement automated firewall blocking
- To generate alerts for suspicious activities
- To use AI for anomaly detection
- To provide a web-based dashboard for monitoring

System Overview

The system is designed as a **cyber range simulation**, where both attack and defense mechanisms are implemented in a controlled environment.

The system performs the following tasks:

- Fetches malicious IP addresses from a threat intelligence API
- Simulates incoming network traffic
- Compares traffic with known malicious IPs
- Blocks detected threats
- Generates alerts
- Displays all activity in a dashboard

Technologies Used

Technology	Purpose
Python	Core programming
Flask	Web application framework
Requests	API communication
Scikit-learn	AI model implementation
NumPy	Data processing
Chart.js	Graph visualization
HTML/CSS	Frontend design
Kali Linux	Development environment

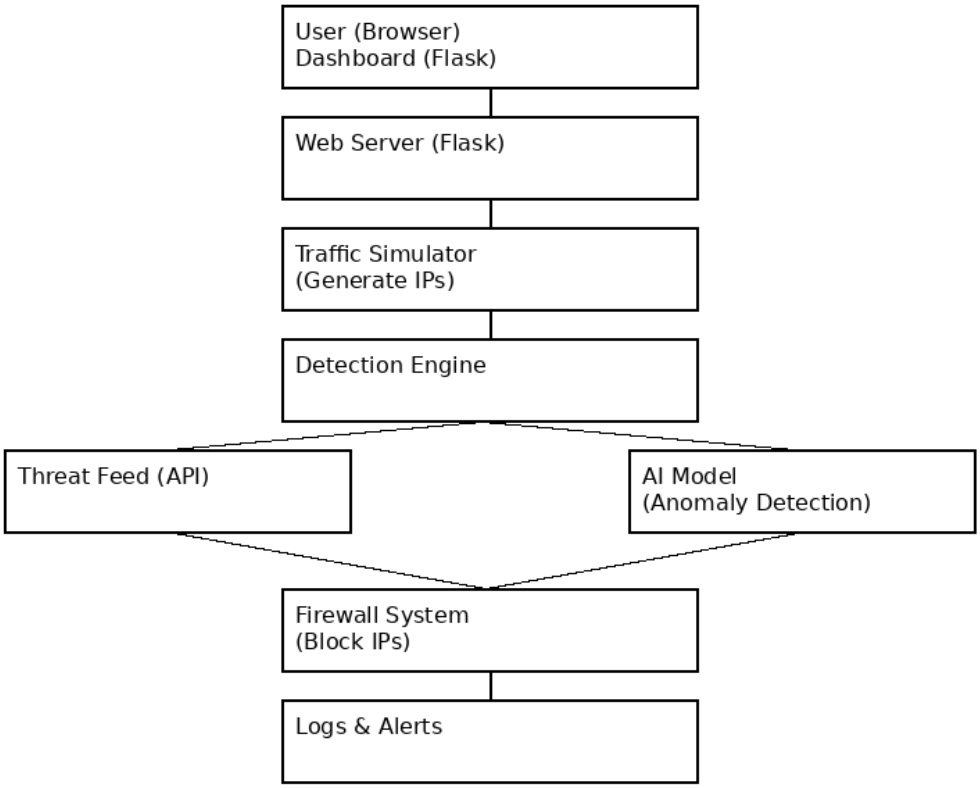
System Architecture

The system consists of multiple components working together:

- **Threat Intelligence Module** → Fetches malicious IPs
- **Traffic Simulation Module** → Generates network traffic

- **Detection Engine** → Identifies threats
- **AI Module** → Detects anomalies
- **Firewall Module** → Blocks malicious IPs
- **Dashboard Module** → Displays logs and statistics

Flow Chart



Methodology

Step 1: Threat Intelligence Integration

The system connects to the AbuseIPDB API and retrieves a list of malicious IP addresses. These IPs are stored in the system for comparison.

Step 2: Traffic Simulation

Random IP addresses are generated to simulate network traffic. Some malicious IPs are intentionally injected into the traffic to test detection.

Step 3: Threat Detection

Each incoming IP address is compared with the threat intelligence feed. If a match is found, it is marked as malicious.

Step 4: Firewall Blocking

Once a malicious IP is detected, it is added to the blocked list, preventing further access.

Step 5: AI-Based Detection

A machine learning model (Random Forest) is used to detect abnormal traffic patterns and generate alerts.

Step 6: Dashboard Visualization

All activities including logs, alerts, and statistics are displayed on a web-based dashboard.

Implementation Details

Environment Setup: A Python virtual environment is created to manage dependencies. Required libraries such as Flask, Requests, and Scikit-learn are installed.

API Integration: The AbuseIPDB API is used to fetch real-time malicious IP addresses. The system filters high-confidence threats for accuracy.

Traffic Simulation: The system generates both safe and malicious traffic to simulate real-world scenarios.

Detection Engine: The detection engine compares incoming traffic with stored malicious IPs and identifies threats.

Firewall Mechanism: Detected malicious IPs are automatically blocked and stored in a block list.

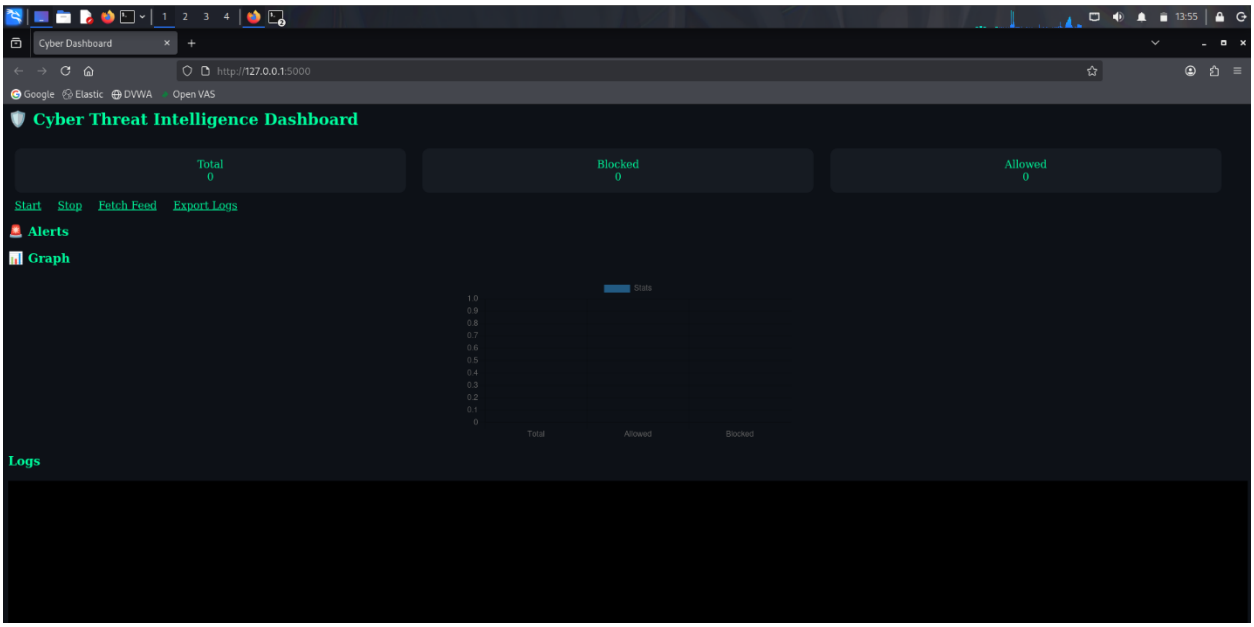
AI Model: A Random Forest model is trained on sample data to identify abnormal traffic behavior.

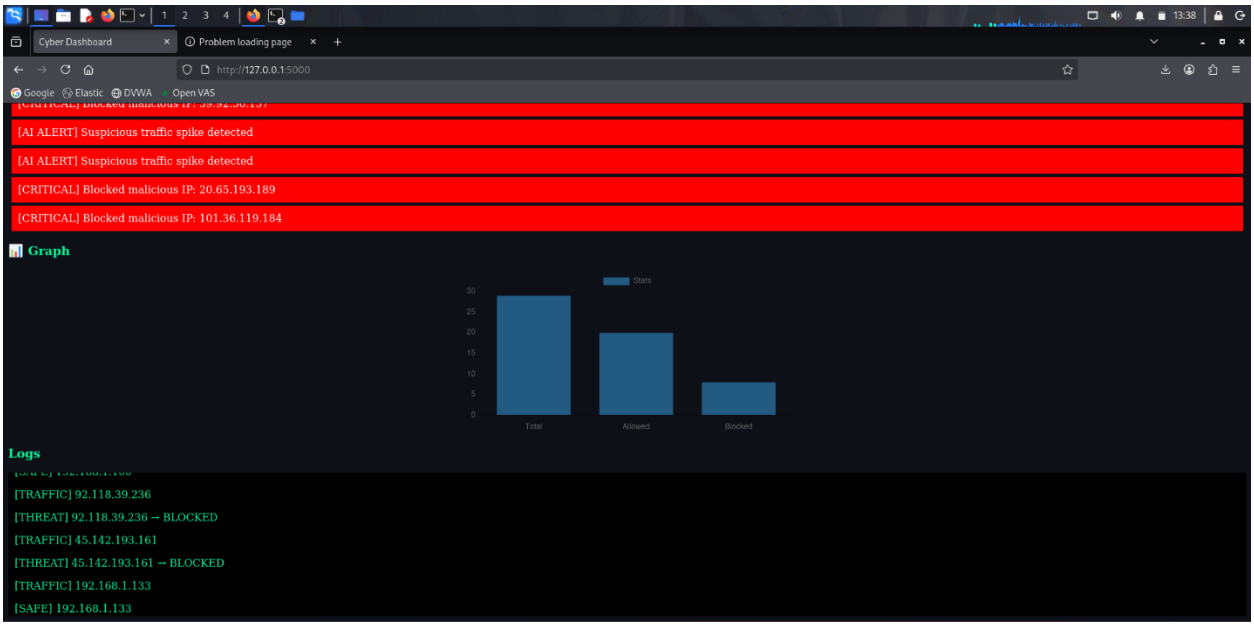
Dashboard: A Flask-based web interface is developed to display logs, alerts, and graphical statistics.

Commands Required to run in kali:

- *cd cyber_dashboard*
- *python3 -m venv venv*
- *source venv/bin/activate*
- *pip install -r requirements.txt*
- *AI&API_Threat detection.py*

Screenshots:





Features

- Real-time threat detection
- Automated firewall blocking
- AI-based anomaly detection

- SOC-style alert system
- Interactive dashboard
- Graphical visualization of traffic

Conclusion

The system demonstrates effective real-time threat detection and response capabilities. When malicious IP addresses appear in the simulated traffic, they are accurately identified and blocked by the firewall mechanism. The AI-based anomaly detection model successfully identifies unusual traffic patterns and generates alerts, enhancing the overall detection capability of the system.

The dashboard provides a clear and interactive visualization of system activity, including traffic statistics, alerts, and logs. This enables users to easily monitor and understand the behaviour of the system. The integration of real-time threat intelligence ensures that the system remains updated with the latest known threats, making it more effective and realistic.

Git hub link: https://github.com/Abhims898/Live-Threat-Intelligence-Feed-Simulator-with-AI-Based-Detection-and-Automated-Firewall

****